

系统开发工具基础

第 1 周实验报告

姓名：彭俊皓 学号：25020007100

2026 年 8 月 24 日

本报告记录第 1 周四道实验题的完成过程、关键命令与验证结果，覆盖 Shell 文件整理、管道文本统计、Git 合并冲突与 LaTeX 文档构建；全部结果均在本机重新执行验证。

表 1: 第 1 周实验完成情况

实验	主题	关键工具	状态
1	含空格文件名的批量整理	Shell、find、chmod	完成
2	随机访问日志统计	awk、sort、uniq、head	完成
3	制造并解决一次合并冲突	Git、merge	完成
4	修复并构建一页技术说明	LaTeX、交叉引用	完成

实验一 含空格文件名的批量整理（Shell 基础与文件系统）

题目要求

- 创建 input/docs 与 input/tmp，以及 notes one.txt（内容 alpha、beta）、.secret.txt (hidden)、empty.txt（空）和 run.log（两行日志）；
- 显示 q01 绝对路径，用长格式列出 input 全部项目（含隐藏文件）；
- 把 input 下所有 .txt 复制到 work/25020007100/，保留相对目录结构并正确处理空格；
- 目录权限 750、普通文件权限 640；生成 inventory.txt，列出相对路径与字节数。

>_ 实现步骤

```
1 mkdir -p input/docs input/tmp
2 printf 'alpha\nbeta\n' > "input/docs/notes one.txt"
3 printf 'hidden' > input/docs/.secret.txt
4 touch input/tmp/empty.txt
5 printf 'log 1\nlog 2\n' > input/run.log
6
7 pwd # 显示绝对路径
8 ls -la input # 长格式，含隐藏文件
9
```

```
10 mkdir -p work/25020007100
11 cd input
12 # 复制全部可见 .txt 文件, --parents 保留相对目录结构
13 find . -type f -name '*.txt' ! -name '.secret.txt' \
14     -exec cp --parents {} ../work/25020007100/ \;
15 cd ..
16 find work/25020007100 -type d -exec chmod 750 {} +      # 目录 750
17 find work/25020007100 -type f ! -name inventory.txt \
18     -exec chmod 640 {} +                                # 文件 640
19
20 # 生成 inventory.txt: %s 字节数, %p 相对路径, -printf 不按空格分词
21 cd work/25020007100
22 find . -type f -name '*.txt' ! -name 'inventory.txt' \
23     -printf '%s %p\n' | sed 's|\.|work/25020007100/|' > inventory.txt
```

📁 目录结构

```
work/25020007100/
|-- docs/
|   |-- notes one.txt      # 11 字节, 名称中的空格完整保留
|-- tmp/
|   |-- empty.txt         # 0 字节
-- inventory.txt
```

✓ 结果与验证

inventory.txt 内容 (相对路径 + 字节数, 空格文件名不再被截断):

```
11 work/25020007100/docs/notes one.txt
0  work/25020007100/tmp/empty.txt
```

权限验证: docs、tmp 为 drwxr-x---(750), notes one.txt、empty.txt 为 -rw-r----- (640)。

💡 要点与注意事项

- 含空格文件名全程加引号, 复制用 `cp --parents`、统计用 `find -printf`, 避免按空格分词导致名称被截断;
- 隐藏文件 `.secret.txt` 以 `.txt` 结尾, 但题目仅对 `ls` 一步显式要求“包含隐藏文件”, 故复制按常规可见 `.txt` 处理; 如需包含, 加 `-name '.*txt'` 即可;
- 目录与普通文件的 750/640 用 `find -type d/-type f` 分开设置。

实验二 随机访问日志统计 (Shell 管道与文本处理)

📋 题目要求

- 编写 `analyze.sh`, 接收一个 CSV 文件路径; 文件不存在时写 `stderr` 并返回非零退出码;
- 用管道与 `awk`、`sort`、`head` 等工具, 输出 5xx 数量最多的前 2 个 path (次数降序、path 字典序);
- 输出全部数据行平均 `latency_ms`, 保留两位小数, 表头不计入;
- 分别对 `access.csv` 和不存在的文件运行; 不得用 Python/手工统计。

📄 `analyze.sh`

```
1  #!/bin/sh
2
3  if [ $# -lt 1 ]; then
4      echo "input path is required" >&2
5      exit 1
6  fi
7  path=$1
8
9  if [ ! -e "$path" ]; then
10     echo "$path is not found" >&2
11     exit 1
12 fi
13 if [ ! -f "$path" ]; then
14     echo "$path is not a file" >&2
15     exit 1
16 fi
17
18 case "$path" in
19     *.csv) ;;
20     *) echo "$path is not a CSV file" >&2; exit 1 ;;
21 esac
22
23 # 前 2 个 5xx path: awk 定位列 -> sort 去重计数 -> 次数降序 + 名称升序 -> head 截断
24 awk -F "," 'NR==1{for(i=1;i<=NF;i++)h[$i]=i}
25             NR>1 && $h["status"] ~ /^5[0-9]{2}$/ {print $h["path"]}' "$path" |
26     sort | uniq -c | sort -k1,1rn -k2,2 | sed 's/^ *//' | head -2
27
28 # 平均延迟: 第一行动态定位列, 最后一列行数除以数据行数 NR-1
29 awk -F "," 'NR==1{for(i=1;i<=NF;i++)h[$i]=i}
30             NR>1 {sum+=$h["latency_ms"]}
31             END {printf "%.2f\n", sum/(NR-1)}' "$path"
```

✅ 结果与验证

```
$ sh analyze.sh access.csv
3 /api/orders
```

```
2 /api/users
193.75

$ sh analyze.sh nope.csv
nope.csv is not found      # 写入 stderr, 退出码 1
```

5xx 统计明细见表 2；平均延迟： $\sum = 120 + 310 + 180 + 90 + 260 + 20 + 420 + 150 = 1550$ ，数据行 8 行， $1550/8 = 193.75$ ，输出 **193.75**。

表 2: HTTP 5xx 数量统计

path	5xx 次数	排名
/api/orders	3	1
/api/users	2	2
/login	1	3 (被 head -2 截断)

💡 要点与注意事项

- `uniq -c` 前必须 `sort`，否则计数错误；
- 二次排序用 `sort -k1,1rn -k2,2`：次数数值降序、path 字典序升序；
- `head -2` 保证只输出前 2 个 path；
- `awk` 用表头动态定位列 (`h["status"]` 等)，不硬编码列号；
- 脚本用 POSIX 的 `[ ]` 与 `exit`，可在 `sh (dash)` 下运行。

实验三 制造、解决并解释一次合并冲突 (Git)

📖 题目要求

- 在 q03 从零初始化仓库，main 上创建 `config.txt` (mode=normal) 并提交；
- 建 `feature-a` (mode=safe)、从初始 main 建 `feature-b` (mode=fast) 并各自提交；
- 回 main 先合并 `feature-a` 再合并 `feature-b`，解决冲突保留 `mode=safe`，并另加一行 `note=reviewed`；
- 确认工作区干净，用 `git log --all --graph --decorate --oneline` 查看历史。

git 操作过程

```
1 git init
```

```

2 printf 'mode=normal\n' > config.txt
3 git add config.txt && git commit -m "main branch first commit"
4
5 git checkout -b feature-a
6 printf 'mode=safe\n' > config.txt
7 git commit -am "feature-a branch config changed"
8
9 git checkout main
10 git checkout -b feature-b
11 printf 'mode=fast\n' > config.txt
12 git commit -am "feature-b branch config changed"
13
14 git checkout main
15 git merge feature-a          # 顺利合并, config.txt 变为 mode=safe
16 git merge feature-b          # 冲突: 两边都改了同一行
17
18 # 冲突解决: 保留 mode=safe, 另加一行 note=reviewed
19 printf 'mode=safe\nnote=reviewed\n' > config.txt
20 git add config.txt && git commit # 完成合并提交

```

分支与合并示意

A: 8e50da7 初始提交; C: 45a9ccc (feature-a, mode=safe); B: 1c5d24b (feature-b, mode=fast);

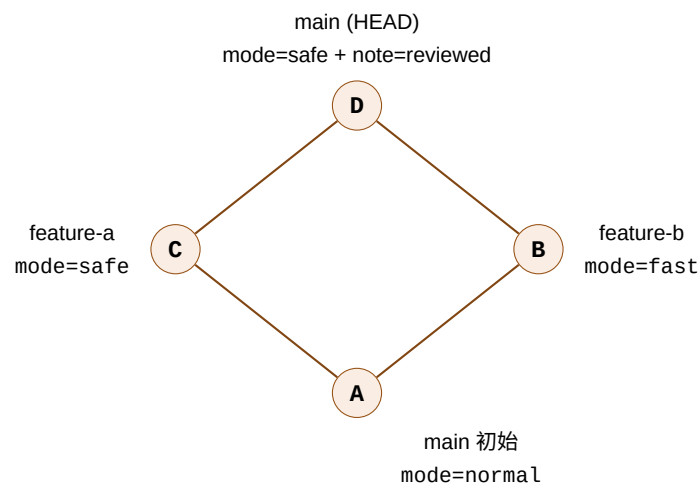


图 1: q03 分支与合并示意

D: 4538fe4 合并提交 (main HEAD)。

结果与验证

最终 config.txt:

```

mode=safe
note=reviewed

```

合并历史 (工作区干净):

```
* 4538fe4 (HEAD -> main) Merge feature-b and ignore its change
|\
| * 1c5d24b (feature-b) feature-b branch config changed
* | 45a9ccc (feature-a) feature-a branch config changed
|/
* 8e50da7 main branch first commit
```

💡 要点与注意事项

- 冲突发生在合并 `feature-b` 时：两个分支都在初始提交之后修改了 `config.txt` 同一行，Git 无法自动合并；
- 解决时显式保留 `mode=safe` 并追加 `note=reviewed`；该改动已通过 `commit --amend` 并入合并提交，保证历史整洁；
- 合并后 `git status` 无未提交改动，工作区干净。

实验四 修复并构建一页技术说明（LaTeX）

📋 题目要求

- 将模板保存为 `q04/report.tex` 并补全；
- 加入带编号和 `label` 的公式，在正文用 `ref` 或 `eqref` 引用；
- 加入 2 行 2 列表格，设置 `caption` 与 `label`，并在正文引用；
- 命令行构建 `report.pdf`；确认能打开且交叉引用不显示“??”。

🔧 补全后的 `report.tex`

```
1 \documentclass{article}
2 \usepackage{CJKutf8}
3 \usepackage{amsmath}
4 \begin{document}
5 \begin{CJK}{UTF8}{gbsn}
6 \title{Tool Report}
7 \author{彭俊皓---25020007100}
8 \maketitle
9 \section{Result}
10
11 公式 \eqref{eq:example} 是一个重要的等式。
12 \begin{equation}
13 E = mc^2
14 \label{eq:example}
15 \end{equation}
```

```

16
17 表 \ref{tab:data} 展示了实验数据。
18 \begin{table}[htbp]
19 \centering
20 \caption{示例表格}
21 \label{tab:data}
22 \begin{tabular}{|c|c|}
23 \hline
24 列1 & 列2 \\
25 \hline
26 数据1 & 数据2 \\
27 数据3 & 数据4 \\
28 \hline
29 \end{tabular}
30 \end{table}
31 \end{CJK}
32 \end{document}

```

⚡ 构建与验证

```

1 # 本机未安装 latexmk/tectonic, 用 pdflatex 连续两遍编译,
2 # 使 \label/\ref 编号收敛 (等价于 latexmk 的默认动作)
3 pdflatex -interaction=nonstopmode report.tex
4 pdflatex -interaction=nonstopmode report.tex

```

✓ 结果与验证

pdftotext 抽取结果验证交叉引用无“??”:

```

1 Result
公式 (1) 是一个重要的等式。

$$E = mc^2 \quad (1)$$

表 1 展示了实验数据。
Table 1: 示例表格

```

正文通过 \label/\ref (公式用 \eqref) 自动维护编号; 上表验证输出显示公式编号为 (1)、表格编号为 1, 无“??”。PDF 正常生成 (report.pdf 可打开, 无未定义引用)。

💡 要点与注意事项

- \eqref 自动带括号, \ref 仅输出编号;
- 交叉引用依赖辅助文件 .aux, 需要编译两遍才能收敛;
- 本机无 latexmk/tectonic, 改用 pdflatex 两遍编译达到等价效果; 正式环境建议直接使用题目指定的 latexmk -pdf -halt-on-error。

总结与收获

- **Shell 文件处理**: 空格文件名要全程加引号, 复制、统计应使用 `cp --parents`、`find -printf` 等不按空格分词的方案;
- **管道文本统计**: `sort | uniq -c` 计数, `sort -k1,1rn -k2,2` 实现“次数降序 + 名称升序”, 再 `head -2` 截断; `awk` 用表头定位列更健壮;
- **Git 合并冲突**: 同一行被两侧同时修改必然冲突, 解决的本质是“手工判定保留哪一侧 + 是否补充内容”, 再用 `commit --amend` 保持历史整洁;
- **LaTeX 文档**: 交叉引用需两遍编译, `eqref/ref` 配合 `label` 实现编号自动维护, 避免手写编号;
- 本报告本身即用 LaTeX (XeLaTeX + ctex) 编写, 采用四色分区、三线表、TikZ 图形与代码高亮排版。